

Resumo AA 3

Comitês de Peritos (Aprendizagem por Ensemble)

Conceitos Fundamentais e Motivação

- **Motivação:** Baseada no teorema do "no free lunch", que indica que não existe um algoritmo com o melhor desempenho para qualquer conjunto de dados.
- **Ideia Central:** Combinar vários modelos (peritos) para criar modelos finais mais poderosos.
- **Criação de Modelos:** A diversidade pode ser alcançada utilizando diferentes algoritmos, diferentes fontes de informação ou diferentes conjuntos de treino.

Combinação e Decisão

- **Múltiplos Peritos (Paralelo):** Cada instância é apresentada a todos os peritos, que produzem uma decisão. Um módulo agregador reúne as respostas para a decisão final.
- **Múltiplas Etapas (Série):** Os modelos trabalham sequencialmente. Cada perito pode dar pesos diferentes aos exemplos ou focar-se num subconjunto.
- **Decisão Final:**
 - **Votação Simples:** Todos os peritos têm o mesmo peso.
 - **Votação Pesada:** Utiliza probabilidades à posteriori ou pondera o desempenho dos peritos.

Estratégias de Comitês

- **Bagging (Bootstrap Aggregating):**
 - Método de múltiplos peritos (paralelo).
 - Geração de peritos: Cria L conjuntos de treino através do método *bootstrap* (retirada aleatória com reposição de N exemplos, podendo haver exemplos repetidos ou ausentes).
 - Agregação: Votação simples dos L peritos.
 - **Desempenho:** Reduz a variância do modelo (sensibilidade a pequenas mudanças no treino). É benéfico para algoritmos instáveis (exemplo: árvores de decisão) e robusto para dados com ruído.
- **Stacking (Stacked Generalization):**
 - Método de múltiplos peritos (paralelo).
 - Agregação: A combinação dos peritos é aprendida. Um agregador é treinado usando um conjunto de dados não utilizado na geração dos peritos base.
 - Requisito: Os peritos devem produzir previsões diferentes, usando algoritmos ou representações distintas.
 - **Objetivo:** Aumentar a generalização do modelo.

- **Boosting:**

- Método de múltiplas etapas (série).
- Geração de peritos: Cada modelo é treinado baseado nos erros dos classificadores anteriores.
- **Características:** Utiliza *weak learners* (aprendizes fracos, como árvores de decisão de profundidade 1). Reduz o enviesamento do modelo (erro devido a falsas suposições).
- **Desvantagens:** Suscetível ao ruído e *outliers*, e pode sofrer de sobre-ajustamento.
- **AdaBoost (Adaptive Boosting):** Utiliza sempre o mesmo conjunto de treino. A votação é pesada pelo desempenho de cada classificador base.

- **Cascading:**

- Método de múltiplas etapas (série).
- Ordenação: Peritos ordenados por complexidade (tempo/espaco ou custo de representação).
- Base lógica: Peritos iniciais (mais simples) tratam da maioria. Peritos mais complexos tratam de uma pequena percentagem.
- Confiança: Cada perito p_j associa uma confiança w_j . Se $w_j > \theta_j$, o perito é confiável.
- Treino: O perito p_{j+1} é treinado apenas com os exemplos para os quais o perito p_j não foi confiável.
- **Uso:** Em cenários onde "não se podem" cometer erros.

Comités de Árvores de Decisão

- **Ideia:** O poder preditivo individual das árvores é mantido, mas o sobre-ajustamento diminui ao agregar os resultados. Cada árvore é diferente das restantes.
- **Algoritmos Chave:** Random Forest, Extremely Randomized Trees (Extra Trees) e Gradient Boosting Trees.

- **Random Forest:**

- Estratégia *bagging*.
- Diversidade: Introdução de aleatoriedade na seleção dos exemplos (*bootstrap*) e na seleção dos atributos no teste de partição.
- Construção: Na criação de um nó, é escolhido aleatoriamente um subconjunto de atributos, pesquisando-se a melhor partição apenas entre eles.
- Previsão: Estratégia de votação "suave" (probabilidades) ou votação simples.
- **Vantagens:** Modelo poderoso, robusto, paralelizável, funciona bem em conjuntos grandes e não requer escala de dados.
- **Desvantagens:** Não funciona bem em dados com muitas dimensões e esparsos.
- **Hiper-parâmetros:** Número de árvores (mais árvores = mais robustez, mas mais memória/tempo) e número de atributos a testar (regra do "polegar": raiz quadrada do número de atributos).

- **Extremely Randomized Trees (Extra Trees):**

- Diversidade: Aleatoriedade na seleção dos atributos e nos pontos de corte (para atributos contínuos).
- **Eficiência:** Menos pesado computacionalmente que Random Forest, pois os pontos de corte são escolhidos aleatoriamente.

- **Gradient Boosting Machines (GBM):**

- Estratégia *boosting*.
- Construção: Árvores sequenciais, cada uma corrigindo os erros da anterior. Usam forte pré-poda (profundidade baixa, ex: 1 a 5).
- **Hiper-parâmetros:** Número de árvores, profundidade da árvore e **taxa de aprendizagem** (controla a força de correção). Uma taxa de aprendizagem menor implica mais árvores para um modelo de complexidade semelhante.
- **Vantagens:** Muito poderoso, muito usado, funciona bem em mistura de dados, não requer escala. Muitas vezes superior ao Random Forest se bem afinado.
- **Desvantagens:** Necessita de afinação cuidadosa dos hiper-parâmetros, tempo de treino longo, pode não funcionar bem em dados esparsos de grande dimensão.
- **Comparação RF/GBM:** GBM é menos pesado computacionalmente devido à menor profundidade das árvores, mas é mais sensível à definição dos hiper-parâmetros (menos aleatoriedade).

Redes Neurais

Introdução e Aplicações

- **Inspiração:** Forte interligação e processamento altamente paralelo do cérebro humano.
- **Tipos Comuns:** Rede Neuronal Feed-Forward (Perceptrão, multi-camada), Rede Neuronal Convolutacional, Autoencoder, Rede Neuronal Recorrente (LSTM), etc.
- **Áreas de Aplicação:** Aprendizagem de informação sensorial complexa (Imagem, Voz, Texto). Características adequadas incluem dados complexos e com ruído, e onde a explicabilidade não é prioritária.

Perceptrão e Regras de Aprendizagem

- **Modelo Linear:** A saída é a soma pesada dos atributos de entrada (ponderada pelos coeficientes/pesos e o componente de limiar b).
- **Perceptrão:** Modelo linear onde os coeficientes são determinados de forma iterativa, modificando os pesos sempre que um exemplo é mal classificado.
- **Regra do Perceptrão:**
 - Atualização de pesos: $w_i = w_i + \eta(t - o)x_i$, onde η é o ritmo de aprendizagem, t o valor objetivo e o a saída do perceptrão.
 - **Convergência:** Garante a convergência se os exemplos forem linearmente separáveis e o ritmo de aprendizagem for suficientemente pequeno (≤ 0.1).
- **Regra do Menor Erro Quadrático (Regra Delta/Adaline):**
 - Atualiza os pesos de acordo com o erro na saída da componente linear, usando o gradiente para pesquisar o espaço de pesos.
 - **Convergência:** Converge assintoticamente para a hipótese com o menor erro, mesmo quando os dados não são linearmente separáveis ou contêm ruído, se η for pequeno.
- **Descida do Gradiente:** Altera o vetor de pesos na direção que produz a descida mais íngreme na superfície de erro. A descida incremental atualiza os pesos após cada exemplo e pode, por vezes, evitar mínimos locais.

Redes Multi-Camada (MLP) e Backpropagation

- **Topografia:** Camada de entrada (atributos), uma ou mais camadas escondidas e camada de saída.
- **Unidades Escondidas:** Permitem a aprendizagem de funções não lineares e a representação de combinações dos atributos de entrada. O número de unidades é crucial: demasiadas causam memorização, a menos impede a generalização.
- **Funções de Ativação:** Aplicadas à saída da componente linear, são funções não lineares que permitem inibir ou ativar a unidade (ex: *sigmoid*, *tanh*, ReLU).
- **Backpropagation (Retropropagação de Erros):** Processo de atualização de pesos em duas fases:
 1. *Feedforward*: Propagação do sinal de entrada até à saída.
 2. *Backpropagation*: O erro é propagado para trás, ajustando os pesos das ligações com base no erro.
- **Convergência:** Garante a convergência para um **mínimo local** (não necessariamente o global). É prática comum treinar vários modelos com pesos iniciais diferentes.
- **Momento:** Mecanismo de aceleração que permite continuar a atualização mesmo na presença de mínimos locais ou regiões planas, dependendo do gradiente e da atualização anterior.
- **CrITÉrios de TerminaÇão:** Podem incluir número fixo de iterações, erro de treino abaixo de um valor ou, preferencialmente, o **aumento do erro no conjunto de validação** (para prevenir sobre-ajustamento).

Parâmetros e Otimização

- **Normalização:** O pré-processamento cuidadoso dos dados é essencial, visto que otimizadores como *adam* são sensíveis à escala dos dados.
- **Otimizadores:**
 - *sgd* (stochastic gradient descent): Muitos parâmetros para afinar.
 - *adam*: Funciona bem na maioria das situações, mas sensível à escala.
 - *l-bfgs*: Mais robusto, mas mais demorado em redes grandes.
- **Ajuste de Parâmetros:** A prática comum sugere criar uma rede grande o suficiente para sobre-ajustar a tarefa, e depois reduzir o tamanho ou aumentar a regularização (parâmetro α) para obter um modelo robusto.
- **Poder de Representação:** Redes com uma camada escondida podem representar funções booleanas e funções contínuas limitadas. Redes com duas camadas escondidas podem representar funções contínuas.

Aprendizagem Não Supervisionada

Conceitos e Desafios

- **Definição:** Área da aprendizagem automática onde não existe um resultado conhecido (não há "professor" ou etiquetas). O algoritmo extrai informação dos dados.
- **Agrupamento (*Clustering*):** Particionar os dados em grupos (clusters) de forma que instâncias do mesmo grupo sejam muito semelhantes e de grupos diferentes sejam pouco semelhantes.
- **Transformação Não Supervisionada:** Criação de uma nova representação dos dados para facilitar a visualização, compressão, ou encontrar representações mais informativas (ex: redução de dimensão).
- **Desafios:** É difícil avaliar o desempenho ou dizer se o algoritmo se "portou bem", uma vez que não existe uma "verdade fundamental" (*ground truth*).

Pré-processamento e Escalas

- **Importância:** Alguns algoritmos (SVM, redes neurais) são sensíveis à escala dos dados, exigindo ajuste por atributo.
- **Transformações Comuns:**
 - **StandardScaler:** Assegura média zero e variância unitária.
 - **RobustScaler:** Utiliza mediana e quartis, ignorando *outliers*.
 - **MinMaxScaler:** Assegura que todos os valores estão entre 0 e 1.
 - **Normalizer:** Escala cada ponto para que a distância Euclidiana seja 1, útil quando apenas a direção é importante.
- **Regra Crítica:** É crucial aplicar a mesma transformação aos conjuntos de treino e teste.

Algoritmos de Agrupamento

- **Agrupamento k-means:**
 - **Ideia:** Pesquisa centros de grupos representativos das regiões dos dados.
 - **Funcionamento:** Alterna entre atribuir cada exemplo ao centro mais próximo e recalculando o centro (média dos dados do grupo) até a convergência.
 - **Características:** O grupo é definido pelo centro. Assume grupos com formas convexas simples e o mesmo "diâmetro". Permite previsão sobre novos exemplos.
 - **Desvantagens:** Depende da inicialização aleatória dos centros e exige a especificação prévia do número de grupos (k).
- **Agrupamento Aglomerativo:**
 - **Funcionamento:** Começa com cada exemplo num grupo e itera juntando os dois grupos mais semelhantes, até uma condição de paragem (ex: número de grupos).
 - **Critério de Ligação (*Linkage*):** Define a semelhança entre grupos. O critério *ward* (menor incremento de variância) é comum.
 - **Características:** Produz um agrupamento hierárquico (visualizado por um Dendograma). Não permite previsão sobre novos exemplos.
- **DBSCAN (Density-Based Spatial Clustering of Applications with Noise):**
 - **Ideia:** Grupos formam regiões densas, separadas por regiões vazias.
 - **Conceitos:** Um **ponto principal** (*core point*) tem pelo menos $min_samples$ vizinhos dentro da distância eps . Os grupos são formados por pontos principais conectados.
 - **Tipos de Pontos Finais:** Pontos principais, Pontos fronteira (vizinhos de pontos principais) e Ruído (que não pertencem a nenhum grupo).
 - **Vantagens:** Não requer a indicação do número de grupos. Consegue capturar grupos com formas complexas. Identifica pontos que são ruído.
 - **Desvantagens:** Mais lento que outros algoritmos. Não permite previsão sobre novos exemplos.
 - **Parâmetros:** eps (distância máxima para ser vizinho, possivelmente o mais importante) e $min_samples$ (tamanho mínimo para formar uma região densa).

Avaliação de Algoritmos de Agrupamento

- **Com Verdade Fundamental (*Ground Truth*):** Utilizam-se métricas como ARI (Adjusted Rand Index) e NMI (Normalized Mutual Information), que variam entre 0 e 1.
- **Sem Verdade Fundamental:**
 - **Coefficiente de Silhueta:** Mede o quão compactos são os grupos (quanto maior, melhor; máximo 1). No entanto, não funciona bem na prática para formas complexas.
 - **Robustez:** O algoritmo é executado após adicionar ruído ou alterar parâmetros. Se o resultado for consistente, é considerado confiável.

Resolução Ficha 10 - Redes Neurais

Enunciado

1. Considere uma rede neuronal usando um perceptrão base. Para a componente de limiar indique quais das seguintes funções $y = f(x)$ podem ser usadas, e justifique a resposta:
 - $y = ax + b$
 - $y = 1/(1 + e^{-x})$ (sigmóide)
 - $y = \tanh(x)$ (tangente hiperbólica)
2. Considere um exemplo descrito por três atributos numéricos $\{x_1 = 0.6, x_2 = 0.2, x_3 = 0.4\}$ pertencente à classe -1 . Se o ritmo de aprendizagem for 0.1 , os pesos do perceptrão forem $\{w_1 = -1, w_2 = 0, w_3 = 2, b = 0\}$ e a componente de limiar for a função `sinhal()`, qual o novo valor dos pesos após a apresentação do exemplo?

Resposta

1. Funções de Limiar em um Perceptrão Base

A componente de limiar (função de ativação) num perceptrão base deve ser uma função **não linear** para permitir que a rede aprenda funções complexas, se for estendida a redes multi-camada.

- **$y = ax + b$: Não pode ser usada.**
 - É uma função linear. Se usada como ativação, a rede continua linear, perdendo capacidade de generalização.
- **$y = 1/(1 + e^{-x})$ (sigmóide): Pode ser usada.**
 - É não linear e comum em redes neurais.
- **$y = \tanh(x)$ (tangente hiperbólica): Pode ser usada.**
 - Também é não linear e frequentemente utilizada.

2. Cálculo do Novo Valor dos Pesos

Regra do Perceptrão:

$$w_i^{novo} = w_i^{anterior} + \eta(t - o)x_i$$

Dados:

$$x = \{0.6, 0.2, 0.4\}, \quad w = \{-1, 0, 2\}, \quad b = 0, \quad t = -1, \quad \eta = 0.1$$

Passo 1: Saída linear

$$y = (-1)(0.6) + (0)(0.2) + (2)(0.4) = -0.6 + 0 + 0.8 = 0.2$$

Passo 2: Saída do perceptrão

$$o = \text{sinhal}(0.2) = +1$$

Passo 3: Erro

$$t - o = (-1) - (+1) = -2$$

Passo 4: Atualização dos pesos

$$w_1^{novo} = -1 + 0.1(-2)(0.6) = -1.12$$

$$w_2^{novo} = 0 + 0.1(-2)(0.2) = -0.04$$

$$w_3^{novo} = 2 + 0.1(-2)(0.4) = 1.92$$

$$b^{novo} = 0 + 0.1(-2)(1) = -0.2$$

Novo vetor:

$$\mathbf{w}_{novo} = \{-1.12, -0.04, 1.92, b = -0.2\}$$